Naming conventions for TrentOS APIs

Page Content

Introduction

- As we are moving towards an "API-stable" version of TrentOS 0.9, we have agreed on some naming conventions for this API.
- In a first step, we define the scope/extent of "the TrentOS API", e.g., what modules and headers are part of the API.
- Secondly, we define conventions for each of the modules/headers that is part of the API.

Scope

- Ultimately, we want to have all API header files which a user of the SDK needs to see in one place: seos_core_api/include (which we SHALL rename in a final step).
- Right now, however, header files are more ore less spread out over individual repositories and other places. Here we list all the modules /repositories which we want to offer as public API as well as their respective header files and where they currently reside.
- **Please note**: The only header files we are interested in here are those, which a user of that specific set of API functions would have to include.
- The coloring indicates which of these modules already have **some** header file in seos_core_api/include (= yellow), all headers already there (= green) and no headers there yet (= red).

Module	Owner	Implementation location(s)	Header file(s)	Header location (s)	Comment	Approved by owner	Is TrentOS Core according to Axel Heider
Configuration	Unknown User (matbuh01)	libs/seos_configuration/	seos_config.h	libs/seos_configuration /include/		✓	✓
ChanMux	Unknown User (carpin01)	libs/seos_libs/src/seos/ChanMux/ blocked URLSEOS-781 - Move ChanMux from seos_lib into a dedicated repo IN DEVELOPMENT	ChanMux.h ChanMuxClient .h ChanMuxRpc.h	libs/seos_libs/include /seos/ChanMux		✓	✗ (driver)
Crypto	Unknown User (bendri01)	libs/seos_crypto/	SeosCryptoApi. h SeosCryptoApi _Agreement.h SeosCryptoApi _Cipher.h SeosCryptoApi _Digest.h SeosCryptoApi _Key.h SeosCryptoApi _Mac.h SeosCryptoApi _Rng.h SeosCryptoApi _Signature.h	libs/seos_core_api /include		✓	✓
Partition Manager	Unknown User (sebeck01)	libs/seos_partition_manager/	seos_pm_api.h seos_pm_conf. h seos_pm_datat ypes.h seos_pm.h	libs /seos_partition_manage r/include		✓	✓
Filesystem	Unknown User (sebeck01)	libs/seos_filesystem_core/	seos_fs_api.h seos_fs_conf.h seos_fs_datatyp es.h seos_fs.h SeosFileStrea m.h SeosFileStrea mFactory.h	libs /seos_filesystem_core /include		✓	✓
	Unknown User (sebeck01)	libs/seos_filesystem_fat/				✓	✗ (fs lib)
	Unknown User (sebeck01)	libs/seos_filesystem_spiffs/				✓	✗ (fs lib)
KeyStore	Axel Heider	libs/seos_keystore/	SeosKeyStoreA pi.h + needs more?	libs/seos_core_api/ + libs/seos_keystore /include ?	KeyStore demos seem to include other headers as well	?	✓

Libs	Unknown User (slakwa01)	libs/seos_libs/src/libDebug/	Debug.h	libs/seos_libs/include/libDebug/	We no longer want a separate libDebug, SHALL become part of the logger.	✓	✗ (lib)
	?	libs/seos_libs/src/libIO/	FifoStream.h FileStream.h FileStreamFactory.h InputFifoStream.h Stream.h	libs/seos_libs/include/libIO/	Which headers are we really using?	?	✗ (lib)
	?	libs/seos_libs/src/LibLogs/	FifoLogger.h Log.h LogFifo.h Logger.h SysLog.h	libs/seos_libs/include/LibLogs/	Unknown User (slakwa01) Do you need this?	?	✗ (lib)
	?	libs/seos_libs/src/libMem/	Allocator.h AllocatorSafe.h BitmapAllocator.h Memory.h Nvm.h	libs/seos_libs/include/libMem/		?	✗ (lib)
	?	libs/seos_libs/src/libUtil/	Bitmap.h CharFifo.h FifoT.h managedBuffer.h MapT.h PointerVector.h	libs/seos_libs/include/libUtil/		?	✗ (lib)
	?	libs/seos_libs/src/seos/	AesNvm.h	libs/seos_libs/include/seos/	Are we using the AesNvm? Do we want to continue using it?	?	✗ (lib)
	Unknown User (carpin01)	libs/seos_libs/src/seos/	ProxyNvm.h	libs/seos_libs/include/seos/		✓	✗ (lib)
Logger	Unknown User (slakwa01)	libs/seos_logger/	seos_logger.h	libs/seos_logger/include/	Actually headers SHALL be split into a client, server, and common header.	✓	✓
Network	Thomas Böhm	libs/seos_network_driver_tap_proxy/	seos_api_chanmux_nic_drv.h	libs/seos_network_driver_tap_proxy/include/		✓	✗ (driver)
	Thomas Böhm	libs/seos_nwstack/	seos_nw_api.h seos_api_network_stack.h	libs/seos_core_api/ libs/seos_nwstack/include	We only want socket API exposed.	✓	✓
TLS	Unknown User (bendri01)	libs/seos_tls/	SeosTlsApi.h	libs/seos_core_api/include		✓	✓

Conventions

- The purpose of having conventions for the trentOS API is to make using different modules easy and also to give a "good feeling" about the state of trentOS. Towards these ends, the API SHALL look sufficiently uniform.
- Here are some aspects that were considered for the creation of this proposal:
  - **Prefix/suffix**: Current API naming schemata often include "API" and "seos" – both are indicating some system-level functionality. In order to reduce redundancy and have a more generic name, it was agreed to use "OS_" as prefix.
  - **Module names**: The API's function names SHALL make it easy to understand structural groupings. For this reason, we embrace this proposal, which found already some adoption in the current trentOS code base.
  - **camelCase vs. snake_case**: Due to the results of this study, we propose to use camelCase as basic naming convention. Specifically, the study says: *"Results indicate that camel casing leads to higher accuracy among all subjects regardless of training, and those trained in camel casing are able to recognize identifiers in the camel case style faster than identifiers in the underscore style."*
- Based on these considerations we make a proposal for naming API-level functions and derive conventions for file and folder names as well as types and defines.

Functions

The following convention **SHALL** be followed when naming API-level functions:

```
OS_<ModuleName><SubModuleName>_<operation>()
```

Here, "<ModuleName>" indicates the functional module the API belongs to and SHALL match the first row of the table above (Crypto, Logger, etc). The "<SubModuleName>" element allows a module to impose another level of grouping, but this is optional. Finally, a function's name SHALL end in "<operation>", which is a short and succinct term (ideally just a verb) describing what the function does. Please note that for the first segment camelCasing applies.

Here is an example how this could look like in practice:

OS prefix	_	Module name	Sub-module name	_	operation	
OS	_	Crypto		_	init	**OS_Crypto_init()**
	_	Crypto	Key	_	generate	**OS_CryptoKey_generate()**
	_	Logger		_	init	**OS_Logger_init()**

	_	Network	ClientSocket	_	connect	**OS_NetworkClientSocket_connect()**

Header and implementation files

In order to make the mapping between an API function name and the respective implementation easy, it makes sense to align those names.

Corresponding to the naming of API functions, implementation and header files **SHALL** follow this convention:

```
    src/OS_<ModuleName>.c
inc/OS_<ModuleName>.h
src/OS_<ModuleName><SubModuleA>.c
inc/OS_<ModuleName><SubModuleA>.h
src/OS_<ModuleName><SubModuleB>.c
inc/OS_<ModuleName><SubModuleB>.h
...
```

CAmkES interfaces

In order to make the mapping between an API functions, implementation files and the respective CAmkES interfaces easy as well, a convention SHALL be followed here as well:

The file name of the interface **SHALL** be as follows:

```
    if_OS_<ModuleName>.camkes
```

The interface name **SHALL** be similarly named:

```
    if_OS_<ModuleName>
```

Notes:

- The "if_" prefix is required, since there also also CAmkES files **`OS_<ModuleName>.camkes`** which describe the module itself.
- A module interface could be different from the actual module name, if the module exposes several interfaces. In this case **`if_OS_<InterfaceName>.camkes`** shall be used for a file that define a specific interface - which can be independent of the actual module that implements it.

Modules vs. folder/repository names

Given that we have a list of module names, it makes sense to not only align header/implementation files but also the names of the repositories and the folder names where these repositories are found in the sandbox/sdk.

The following naming convention **SHALL** be followed for libraries:

```
    <SDK>/libs/<ModuleName>/
```

And analogously this scheme **SHALL** be used for components:

```
    <SDK>/components/<ModuleName>/
```

Note: Some modules (Network, Filesystem) are spread across different directories. They SHALL be consolidated into one single repository which follows the convention.

Typedefs, enums, defines

Furthermore, we want to make it easy to understand which type belongs to which API function and which module/sub-module respectively.

The following convention for typedefs of structs or enums **SHALL** be applied:

```
    typedef struct {
        int a;
        int b;
} OS_<ModuleName><SubModuleName>_<TypeName>_t;
```

This way, a type can be directly mapped to the API and the implementation level (if we follow the file name conventions proposed above).

Further specializations of "_t" are possible and **CAN** be used:

Specialized suffix	Usage
_ptr	For a typedef'd pointer which SHALL be explicitly marked as such
_cptr	For a typedef'd const pointer
_func	For a function pointer

Opaque handles

For all modules that follow the "multi-instance" pattern, which allows a more-or-less "object oriented" style of programming in C, we would like to use "opaque handles". Such a handle can be given out to the users of the API without those users not needing to know what is underneath it; marking it clearly in such a way tells the caller that he SHALL not make assumption about the handle and/or try to de-reference it. Behind the handle can be anything, typically the API module would use it to manage a blob of allocated memory where it stores its context.

To make different modules of the API-layer look as consistent as possible, the following convention for the naming of handles **SHALL** be followed:

```
    OS_<ModuleName><SubModuleName>_Handle_t
```

Assuming we have a module OS_Foo, then the corresponding handle would be OS_Foo_Handle_t.

It makes sense if all functions that follow this pattern provide this context parameter always as the first argument to the respective function (similar to "self" parameter in Python, etc.). This convention **CAN** be applied.

Here are some examples for an init() and free() function:

```
    OS_Foo_init(OS_Foo_Handle_t* hndl, ...);
OS_Foo_free(OS_Foo_Handle_t hndl);
```

If the caller has to provide the memory, a generic init function **SHALL** look like this:

```
    OS_err_t
OS_Foo_init(
        OS_Foo_Handle_t* hFoo
        OS_Foo_t foo,
    ...
);
```

Where the API return OS_Foo_Handle_t as a handle to the OS_Foo_t object. Implementation-wise OS_Foo_Handle_t might just simply be defined as OS_Foo_Handle_t* then in case of a local library.